

**Tugas Besar 2 IF3270 Pembelajaran Mesin
Convolutional Neural Network dan Recurrent Neural Network**

Oleh:

Yosef Rafael Joshua	13522133
Chelvadinda	13522154

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

BAB I

DESKRIPSI PERSOALAN

Pada tugas besar ini dilakukan implementasi *Convolutional Neural Network* (CNN), *Recurrent Neural Network* (RNN), dan *Long Short-Term Memory* (LSTM) menggunakan framework Keras serta implementasi forward propagation from scratch menggunakan NumPy. Pada bagian CNN, model digunakan untuk task image classification menggunakan dataset *Intel Image Classification* yang terdiri dari enam kategori gambar, yaitu *buildings*, *forest*, *glacier*, *mountain*, *sea*, dan *street*. Model CNN dibangun menggunakan beberapa layer seperti Conv2D, pooling layer, flatten/global pooling layer, dan dense layer. Pelatihan model dilakukan menggunakan optimizer Adam dan loss function Sparse Categorical Crossentropy. Selain menggunakan Conv2D dengan shared parameters, pada tugas ini juga dilakukan implementasi LocallyConnected2D sebagai non-shared parameters untuk membandingkan pengaruh parameter sharing terhadap performa model.

Tahapan awal pengerjaan dimulai dengan implementasi utility functions menggunakan PIL/Pillow dan NumPy tanpa preprocessing bawaan Keras. Utility function yang dibuat meliputi *image loader* untuk membaca dan melakukan preprocessing gambar, *batch loader* untuk memproses sekumpulan gambar menjadi numpy array, serta feature extractor yang menggunakan pretrained CNN encoder untuk menghasilkan feature vector dan menyimpannya ke disk. Setelah itu dilakukan implementasi forward propagation from scratch secara modular untuk setiap layer, seperti Conv2D, LocallyConnected2D, pooling layer, flatten layer, dan fungsi aktivasi menggunakan NumPy dengan memanfaatkan bobot hasil pelatihan model Keras.

Pada proses pelatihan CNN dilakukan beberapa eksperimen hyperparameter yang meliputi jumlah convolution layer, jumlah filter pada convolution layer, ukuran filter/kernel, serta jenis pooling layer yang digunakan. Seluruh model kemudian dievaluasi menggunakan macro F1-score, kemudian dilakukan analisis terhadap hasil prediksi, grafik training dan validation loss, serta perbandingan antara arsitektur shared dan non-shared parameter.

Selain CNN, tugas besar ini juga membahas image captioning menggunakan kombinasi CNN encoder dan decoder berbasis RNN dan LSTM pada dataset Flickr8k. CNN encoder pretrained digunakan untuk mengekstraksi feature vector dari gambar, kemudian decoder RNN dan LSTM

digunakan untuk menghasilkan caption teks. Pada bagian ini dilakukan preprocessing caption berupa tokenisasi, pembangunan vocabulary, penambahan special token, serta padding sequence. Selanjutnya dilakukan eksperimen pada arsitektur RNN dan LSTM menggunakan beberapa variasi recurrent layer dan hidden state, kemudian dilakukan evaluasi menggunakan BLEU-4 score dan waktu eksekusi untuk membandingkan performa implementasi Keras dan implementasi from scratch.

BAB II

IMPLEMENTASI

A. CNN

- Utility Function

Nama Fungsi	Parameter	Output	Deskripsi
load_image()	image_path : Path gambar target_size : Ukuran resize gambar	image_array : Numpy array hasil preprocessing	Digunakan untuk membaca gambar dari file path, melakukan resize gambar, mengubah gambar menjadi numpy array, dan melakukan normalisasi pixel
load_balanced_dataset()	dataset_path : Path dataset	X : Data gambar Y: Label gambar class_names : Nama kelas	Digunakan untuk membaca dataset gambar dan menghasilkan batch data dalam bentuk numpy array dengan distribusi kelas yang seimbang

- CNN Model

Nama Fungsi	Parameter	Output	Deskripsi
build_cnn_model()	conv_layers: Jumlah convolution layer filters : Jumlah filter tiap layer kernel_size : Ukuran kernel	model : Model CNN Keras	Digunakan untuk membangun arsitektur CNN secara dinamis berdasarkan hyperparameter yang diberikan.

	pooling_type Jenis pooling	:		
--	-------------------------------	---	--	--

- Training Model

Dibuat training pipeline yang digunakan untuk melakukan training 16 variasi arsitektur CNN menggunakan kombinasi hyperparameter yang berbeda.

Proses:

1. Membuat model CNN
2. Training model
3. Evaluasi model
4. Menyimpan model
5. Menyimpan hasil eksperimen

- Evaluasi Model

Digunakan untuk menghitung precision, recall, dan F1-score untuk setiap kelas serta melihat distribusi prediksi model terhadap label sebenarnya.

- *Forward Propagation*

Forward propagation diimplementasikan secara manual menggunakan NumPy dengan memanfaatkan bobot hasil training terbaik model CNN. Proses *forward propagation* dilakukan dengan menjalankan setiap layer secara berurutan mulai dari input gambar hingga menghasilkan prediksi kelas akhir. Input berupa gambar dengan ukuran $150 \times 150 \times 3$ diproses menggunakan layer Conv2D untuk menghasilkan feature map. Operasi konvolusi dilakukan menggunakan kernel hasil training Keras dengan mekanisme shared parameter. Pada setiap posisi spasial dilakukan perhitungan dot product antara patch input dan kernel, kemudian ditambahkan bias.

Output hasil konvolusi kemudian diproses menggunakan fungsi aktivasi ReLU untuk mengubah nilai negatif menjadi nol. Setelah itu dilakukan AveragePooling2D untuk mengurangi dimensi feature map dan mempertahankan fitur penting. Feature map hasil pooling kemudian diubah menjadi vektor satu dimensi menggunakan Flatten layer. Vektor tersebut diteruskan ke Dense layer untuk menghasilkan representasi fitur sebelum masuk ke output layer.

Pada layer terakhir digunakan fungsi aktivasi Softmax untuk menghasilkan distribusi probabilitas terhadap enam kelas dataset *Intel Image Classification*. Kelas prediksi akhir diperoleh dari nilai probabilitas terbesar.

BAB III

HASIL PENGUJIAN

A. CNN

- Perbandingan shared vs non-shared parameter

```
...
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 16)	448
average_pooling2d (AveragePooling2D)	(None, 74, 74, 16)	0
conv2d_1 (Conv2D)	(None, 72, 72, 32)	4,640
average_pooling2d_1 (AveragePooling2D)	(None, 36, 36, 32)	0
conv2d_2 (Conv2D)	(None, 34, 34, 64)	18,496
average_pooling2d_2 (AveragePooling2D)	(None, 17, 17, 64)	0
flatten (Flatten)	(None, 18496)	0
dense (Dense)	(None, 64)	1,183,808
dense_1 (Dense)	(None, 6)	390

```
...
```

Total params: 3,623,348 (13.82 MB)

```
...
```

Trainable params: 1,207,782 (4.61 MB)

```
...
```

Non-trainable params: 0 (0.00 B)

```
...
```

Optimizer params: 2,415,566 (9.21 MB)

```
...
```

None

Gambar 3.1 CNN menggunakan shared parameter dengan layer Conv2D

Perbedaan utama antara *shared parameter* dan *non-shared parameter* terletak pada penggunaan kernel pada proses konvolusi. Pada *shared parameter* menggunakan Conv2D, kernel yang sama digunakan secara berulang pada seluruh area gambar sehingga jumlah parameter menjadi lebih sedikit dan komputasi lebih efisien. Sedangkan pada *non-shared parameter*, kernel dibuat berbeda untuk

setiap posisi feature map sehingga jumlah parameter dan kebutuhan komputasi menjadi jauh lebih besar. Berdasarkan hasil implementasi, *non-shared parameter* berhasil menghasilkan output feature map berukuran (148, 148, 16) yang sesuai dengan ukuran output Conv2D *shared parameter*. Namun, karena setiap posisi menggunakan kernel berbeda, *non-shared parameter* membutuhkan memori lebih besar dibandingkan *shared parameter*.

- Pengaruh jumlah layer konvolusi

	model	conv_layers	filters	kernel_size	pooling	accuracy	loss
9	10	3	[8, 16, 32]	(3, 3)	avg	0.805873	0.670051
13	14	3	[16, 32, 64]	(3, 3)	avg	0.795957	0.956365
8	9	3	[8, 16, 32]	(3, 3)	max	0.781465	1.137500
14	15	3	[16, 32, 64]	(5, 5)	max	0.780320	1.136779
10	11	3	[8, 16, 32]	(5, 5)	max	0.779558	0.990924
11	12	3	[8, 16, 32]	(5, 5)	avg	0.773074	0.962134
15	16	3	[16, 32, 64]	(5, 5)	avg	0.766972	0.977617
4	5	2	[16, 32]	(3, 3)	max	0.759344	1.321887
5	6	2	[16, 32]	(3, 3)	avg	0.758581	1.376772
6	7	2	[16, 32]	(5, 5)	max	0.754005	1.515244
12	13	3	[16, 32, 64]	(3, 3)	max	0.750572	1.363607
2	3	2	[8, 16]	(5, 5)	max	0.746377	1.609319
0	1	2	[8, 16]	(3, 3)	max	0.741800	1.176336
1	2	2	[8, 16]	(3, 3)	avg	0.737986	1.382486
3	4	2	[8, 16]	(5, 5)	avg	0.733791	1.493161
7	8	2	[16, 32]	(5, 5)	avg	0.709001	1.868099

Gambar 3.2 Hasil training 16 variasi hyperparameter

Berdasarkan hasil pengujian pada Gambar 3.2, variasi jumlah layer konvolusi memberikan pengaruh terhadap performa model CNN. Pengujian dilakukan menggunakan dua variasi jumlah layer konvolusi, yaitu 2 layer Conv2D dan 3 layer Conv2D. Setiap variasi diuji menggunakan kombinasi jumlah filter, ukuran kernel, dan jenis pooling yang berbeda untuk melihat pengaruhnya terhadap *accuracy* dan *loss model*. Berdasarkan tabel hasil eksperimen, model dengan 3 layer konvolusi cenderung menghasilkan performa yang lebih baik dibandingkan model dengan 2 layer konvolusi. Hal ini terlihat pada model 10 yang menggunakan 3 layer konvolusi dengan kombinasi filter [8, 16, 32], kernel (3,3), dan average pooling yang memperoleh *accuracy* tertinggi sebesar 0.805873 dengan *loss* sebesar 0.670051. Selain itu, beberapa model lain dengan 3 layer konvolusi juga menghasilkan *accuracy* yang tinggi dibandingkan model dengan 2

layer konvolusi. Sedangkan pada model dengan 2 layer konvolusi, *accuracy* yang dihasilkan lebih rendah. Hal ini menunjukkan bahwa jumlah layer konvolusi yang lebih sedikit membuat kemampuan ekstraksi fitur model menjadi lebih terbatas.

- Pengaruh banyak filter per layer konvolusi
Dapat dilihat pada gambar 3.2, variasi jumlah filter yang digunakan pada eksperimen ini antara lain [8, 16], [16, 32], [8, 16, 32], dan [16, 32, 64]. Jumlah filter menentukan banyaknya *feature map* yang dihasilkan oleh layer konvolusi. Semakin banyak filter yang digunakan, maka model dapat mempelajari lebih banyak pola dan fitur dari gambar input. Berdasarkan hasil eksperimen, penggunaan jumlah filter yang lebih besar menghasilkan performa klasifikasi yang lebih baik. Hal ini terlihat pada model dengan kombinasi filter [8, 16, 32] dan [16, 32, 64] yang mampu menghasilkan *accuracy* lebih tinggi dibandingkan model dengan jumlah filter lebih sedikit seperti [8, 16]. Model 10 dengan kombinasi filter [8, 16, 32] menghasilkan *accuracy* tertinggi sebesar 0.805873.
- Pengaruh ukuran filter per layer konvolusi
Dapat dilihat pada gambar 3.2, eksperimen dilakukan menggunakan dua variasi ukuran kernel pada layer konvolusi, yaitu (3,3) dan (5,5). Ukuran kernel menentukan luas area gambar yang diproses pada setiap operasi konvolusi. Kernel yang lebih kecil fokus pada detail lokal gambar, sedangkan kernel yang lebih besar dapat menangkap area fitur yang lebih luas. Berdasarkan hasil eksperimen, model dengan ukuran kernel (3,3) cenderung menghasilkan performa yang lebih baik dibandingkan model dengan kernel (5,5). Hal ini terlihat pada model 10 yang menggunakan kernel (3,3) dan memperoleh *accuracy* tertinggi sebesar 0.805873. Selain itu, beberapa model lain dengan kernel (3,3) juga menunjukkan nilai *accuracy* yang lebih tinggi dibandingkan model dengan kernel (5,5)..
- Pengaruh jenis pooling layer
Dapat dilihat pada gambar 3.2, eksperimen dilakukan menggunakan dua jenis pooling layer, yaitu *Max Pooling* dan *Average Pooling*. Berdasarkan hasil eksperimen, penggunaan *Average Pooling* menghasilkan performa yang lebih baik dibandingkan *Max Pooling*. Hal ini terlihat pada model 10 yang menggunakan *Average Pooling* dan memperoleh *accuracy* tertinggi yaitu 0.805873. Selain itu, beberapa model lain dengan *Average Pooling* juga menghasilkan *accuracy* yang lebih tinggi dibandingkan model dengan *Max Pooling*. Perbedaan performa yang dihasilkan menunjukkan bahwa pemilihan jenis pooling layer dapat mempengaruhi hasil klasifikasi model tergantung karakteristik dataset yang digunakan. Berdasarkan hasil eksperimen yang diperoleh, *Average Pooling* memberikan hasil yang lebih optimal.
- Perbandingan Keras vs from scratch


```
[134] print("SCRATCH PREDICTION:")
      print(class_names[predicted_class])

      print("\nKERAS PREDICTION:")
      print(class_names[keras_predicted_class])

... SCRATCH PREDICTION:
    forest

    KERAS PREDICTION:
    forest
```

Gambar 3.3 Hasil Prediksi

Perbandingan dilakukan antara hasil prediksi model CNN menggunakan Keras dan hasil *forward propagation from scratch*. Implementasi tersebut dilakukan dengan memanfaatkan bobot hasil training model terbaik dari Keras, kemudian setiap layer dijalankan secara manual mulai dari Conv2D, ReLU, AveragePooling2D, Flatten, Dense, hingga Softmax. Pada implementasi *Keras*, proses dilakukan secara otomatis menggunakan layer bawaan *TensorFlow/Keras*. Sedangkan pada implementasi *from scratch*, seluruh proses perhitungan dilakukan secara manual menggunakan operasi NumPy. Berdasarkan hasil pengujian, prediksi kelas yang dihasilkan oleh implementasi *from scratch* sesuai dengan hasil prediksi model Keras pada gambar input yang sama. Pada pengujian yang dilakukan, kedua implementasi menghasilkan prediksi kelas forest. Hasil ini menunjukkan bahwa implementasi *forward propagation* manual berhasil merepresentasikan proses CNN dengan benar. Selain menghasilkan prediksi kelas yang sama, ukuran output pada setiap layer *forward propagation from scratch* juga sesuai dengan output layer pada model Keras. Berdasarkan hasil perbandingan tersebut, implementasi *from scratch* sesuai dengan proses *forward propagation* CNN secara manual menggunakan NumPy.

BAB IV KESIMPULAN DAN SARAN

A. Kesimpulan

Berdasarkan hasil implementasi dan eksperimen yang telah dilakukan menunjukkan bahwa variasi hyperparameter memberikan pengaruh terhadap performa model CNN. Penambahan jumlah layer konvolusi dan jumlah filter cenderung meningkatkan kemampuan ekstraksi fitur sehingga menghasilkan *accuracy* yang lebih baik. Selain itu, penggunaan ukuran kernel (3,3) dan *Average Pooling* memberikan hasil yang lebih optimal dibandingkan variasi lainnya pada eksperimen yang dilakukan.

Implementasi *forward propagation from scratch* berhasil menghasilkan prediksi yang sesuai dengan model Keras pada gambar input yang sama. Hasil ini menunjukkan bahwa proses Conv2D, pooling, flatten, dense, serta fungsi aktivasi berhasil diimplementasikan secara manual menggunakan NumPy sesuai dengan arsitektur model CNN dengan menggunakan *keras*.

B. Saran

Implementasi non-shared parameter dapat dilakukan lebih lanjut menggunakan environment *TensorFlow* yang mendukung *LocallyConnected2D* sehingga proses training dan evaluasi non-shared parameter dapat dilakukan dan dibandingkan langsung dengan *shared parameter* menggunakan *accuracy*, *macro F1-score*, dan *confusion matrix*. Selain itu, implementasi yang telah diselesaikan berfokus pada bagian CNN meliputi training model, eksperimen hyperparameter, forward propagation from scratch. Sedangkan bagian lain seperti implementasi *Recurrent Neural Network* (RNN), *Long Short-Term Memory* (LSTM), dan pipeline *image captioning* belum diimplementasikan.